

CSSE 230 – Data Structures and Algorithms

[Typical Exam instructions] **Solution**

You may use a calculator, a writing implement, and three sides of 8.5 x 11 note pages. Blank paper is also available to use as scratch paper. You must turn in this written part before you use your computer, textbook, or any other resources.

You may not communicate with any person other than your instructor about this exam until allowed by your instructor. You may not use for any purpose email, chat, cell phones or any communication device until you have submitted your exam, unless you are given specific permission by your instructor.

Part 2 (computer) rules

You may use any printed or written materials that you brought with you, code that you wrote yourself or with a partner before the exam, anything on the CSSE230 Moodle and web sites, and the Java API documentation (on your laptop or at <http://download.oracle.com/javase/7/docs/api/>).

You may not otherwise search the internet or communicate with any person other than your instructor or a student assistant. You may not communicate with anyone else about this exam until allowed by your instructor.

With my signature, I certify that I neither received help from others nor gave it on either part of the exam: _____

You must actually get this problem working on your computer. Almost all of the credit for this problem will be for code that actually works.

At the end of the computer part, you will find this checklist.

Please check off each item below:

Turn-in checklist:

1. ☐ I signed the statement above.
2. ☐ I saved my work.
3. ☐ I committed my solution via SVN.
4. ☐ I saw in Eclipse that my solution was committed
5. ☐ I circled the number of the programming problem I want graded (if I was given a choice)
6. ☐ I am about to hand this test paper to my instructor or an assistant.

1. (10 points) When writing HeapSort, we discussed two different approaches to the sub problem of initially arranging elements 1-N of an array into a binary heap, and chose one of them because it was more efficient. Briefly describe the two approaches and explain why one is more efficient than the other.

The first one is to start with a one-element heap and insert each element of the array (from position 2 to position N-1 into the heap in turn. When inserting element k, the heap already has k-1 elements, so the worst-case time for insertion is the height of node k, which is approximately $\log k$.

Thus the total worst-case time for building the heap is the sum of the depths of the nodes, $\sum(\log k, k=2..N)$, which is $\log(N!)$, whose value is approximately $N \log N$.

The second approach (called BuildHeap in the textbook) is "bottom-up". All of the leaves of the tree already have the heap property. (The leaves are in array positions beyond N/2. For every node with children, all we must do to make its subtree become a heap is "percolate it down", as in the deleteMin algorithm. So the worst case total number of comparisons turns out to be the sum of the heights of the nodes. This sum is less than N, and thus considerably smaller than the result from the previous paragraphs.

2. (10 points)

Consider the recurrence
 $T(n) = aT(n/b) + f(n)$, $T(1)=c$,
where $f(n) = \Theta(n^k)$ and $k \geq 0$,

The solution is

- $\Theta(n^k)$ if $a < b^k$
- $\Theta(n^k \log n)$ if $a = b^k$
- $\Theta(n^{\log_b a})$ if $a > b^k$

Write and solve (big-theta) a recurrence relation for the running time of the following meth

```
public static void mystery(int n) {  
    if (n==1)  
        return;  
    mystery(n/10);  
    mystery(n/10);  
    for (int i=1; i<= n; i++)  
        nTimeMethod(i, n); // this function call always takes O(n) time.  
    mystery(n/10);  
}
```

2 points for correct equation but incorrect solution.

$$T(n) = 3 T(n/10) + n^2, T(1) = 1.$$

$a = 3, b = 10, k = 2$. Thus $a < b^k$ so it is the first of the three cases, and thus $T(N) = \Theta(N^2)$.

3. (16 points) Someone writing a class to implement the **priority queue ADT** has many options of underlying data structures to use to contain the data. Briefly discuss the advantages and disadvantages of each proposed representation, in terms of the three **PriorityQueue** operations (insert, findMin, deleteMin). You should consider big-theta runtime of the main operations, programming ease, and memory requirements.

throughout: fast = $\log n$, slow = N 3 points for each row.

Take away ½ or 1 point for each thing that is missing or incorrect.

Structure	Advantage(s)	Disadvantage(s)
Unsorted array	insert is fast	find and delete are slow $O(n)$
Sorted array	findMin and deleteMin are $O(1)$ assuming sorted in reverse order	insert is slow: $O(n)$
Sorted linked list	Same as sorted array.	Same as sorted array. Extra space for Node info.
Binary Search Tree	Average time for find, insert, delete is $O(\log N)$	If not balanced, all operations can be $O(N)$. Extra space for Node info.
AVL tree	Worst-case for find, insert, delete all $O(\log N)$	Algorithms are complicated. Extra space for Node info.
Hash Table	Insert is $O(1)$	Not sorted at all, so slow to find or delete min, $O(n)$ each
Binary Heap	Insert and deleteMin are $O(\log n)$, and findMin is $O(1)$	None. It's the preferred implementation for a priority queue.

4. (15 points) In the EditorTrees project, you were given this definition of a node in a height-balanced binary tree with rank::

```
public class Node {  
    enum Code {SAME, LEFT, RIGHT};  
  
    char element;  
    Node left, right; // subtrees  
    Node parent;  
    int rank;          // inorder position of this node within its own subtree.  
    Code balance;
```

Having a balance code allowed us to determine which node in an AVL tree was the deepest node that became unbalanced (thus needed a rotation) as a result of inserting a single element. Please give pseudocode for an algorithm for doing this that operates in $O(\log n)$ time. (Note: pseudocode can use a mixture of Java and English, but is still precise).

Assumptions:

You have a reference to the node that was just inserted.

The balance codes have not yet been changed in the process of inserting the new node.

For this problem, you do not need to modify any balance codes or do any rotations.

Your pseudocode should output

NO ROTATION, STOPPED AT NODE WITH ELEMENT x	if no rotations are required, or
ROTATION AT NODE WITH ELEMENT x	if a rotation is required.

where x is the actual element of the node.

This solution also modifies balance codes.

Start at the inserted node n and mark it =. Then loop:

Check n's parent. If it has none (n is the root), stop and output NO ROTATION, STOPPED AT NODE WITH ELEMENT n.element.

If n's parent was tipped away from n (e.g., if n is a right child, but parent's balance was LEFT), set n's parent's code to =, stop and output NO ROTATION, STOPPED AT NODE WITH ELEMENT parent.element.

If n's parent was =, tip n's parent towards n and continue.

If n's parent was tipped toward n already, stop and output ROTATION AT NODE WITH ELEMENT parent.x.

Set n to n.parent

This algorithm is $O(\log n)$ because we only consider at most a number of nodes equal to the height of the tree and the height of any AVL tree is $O(\log n)$.

5. (12 points) Consider a very simple hashcode function for integers, $h(N) = N$. The following list of numbers is inserted (in the order given) into an initially empty table. Show where each number ends up in this table of size 12 if
- (a) linear probing, and (b) quadratic probing is used. Most of the points will be for correctly dealing with collisions.

16, 3, 29, 7, 15, 24, 11, 22, 4

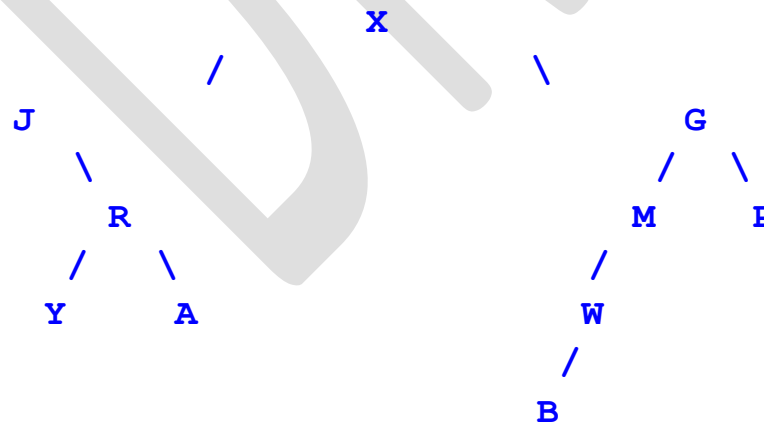
i:0	1	2	3	4	5	6	7	8	9	10	11
24			3	16	29	15	7	4		22	11
15	24		3	16	29		7	4		22	11

6. (10 pts) A Binary Search Tree whose elements are characters has the following properties:

(a) A level-order traversal of its ten nodes yields: **X J G R M P Y A W B.**

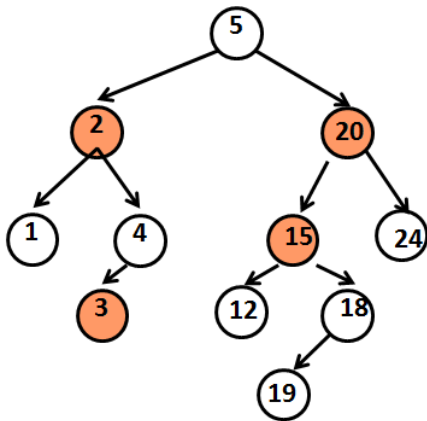
(b) An in-order traversal its ten nodes yields: **J Y R A X B W M G P.**

Draw the tree.



No partial credit if not close

7. (6 pts) Is this a valid red-black BST or not? (The shaded nodes are red. The null external nodes aren't shown.) Hint: it violates three of the five red-black tree properties – which ones?



1. 15 and 20 are 2 successive red nodes.
2. All paths from the root to leaves don't have the same number of black nodes (the path to 19 has one more)
3. It is not a BST, since 19 is 18's left child.

8. (10 pts) Write code that given a node n, gets the node's successor in an in-order traversal of the tree, or null if it is the last node in an in-order traversal. Assume that each node has a pointer to its parent:

```

class Node {
    int element;
    Node left;
    Node right;
    Node parent;
    ...
}

public Node getInorderSuccessor(Node n) {
    // If it has a right child, the succ is the left-most node in the right subtree.
    if (n.right != null) {
        n = n.right;
        while (n.left != null) {
            n = n.left;
        }
        return n;
    }
    // If it has no right child, then n is the rightmost child of some subtree, so go to the parent of that subtree.
    while (n.parent != null && n.parent.left != n) {
        n = n.parent;
    }
    return n.parent; // will be null if n is the root (that is, if the node has no successor)
}
  
```

9. (10 pts) Given two sets A and B represented as sorted lists (your choice of ArrayList or LinkedList), consider an algorithm for computing $A \oplus B$ (the set of elements that are in A or B, but not both).
- a) Write code or pseudocode (Java syntax not required, as long as you are precise) to do this efficiently.

```

List ans = new ArrayList();
int aIdx = bIdx = 0;
while (aIdx < A.size() && bIdx < B.size()) {
    if (A.get(aIdx) < B.get(bIdx)) {
        ans.add(A.get(aIdx));
        aIdx++;
    } else if (A.get(aIdx) > B.get(bIdx)) {
        ans.add(B.get(bIdx));
        bIdx++;
    } else { // both are equal
        aIdx++; bIdx++;
    }
}
// Now we copy the rest of the array that we haven't yet traversed fully
if (aIdx < A.size()) {
    for (int i = aIdx; i < A.size(); i++) {
        ans.add(A.get(i));
    }
} else {
    // repeat for b (analagous)
}

```

b) What is the runtime of your algorithm? $\Theta(\underline{n})$

10. (5 pts) Suppose we call **QuickSort** on the array below. Assume 45 is chosen as the pivot. Show the array contents after a partition is done, but before the recursive calls.

11 18 81 9 55 10 77 98 45 50 78 22 38

(All swaps shown)

(45)	18	81	9	55	10	77	98	(11)	50	78	22	38 (pivot to start)
45	18	(38)	9	55	10	77	98	11	50	78	22	(81) (swap 1)
45	18	38	9	(22)	10	77	98	11	50	78	(55)	81 (swap 2)
45	18	38	9	22	10	(11)	98	(77)	50	78	55	81 (swap 3)
(11)	18	38	9	22	10	(45)	98	77	50	78	55	81 (pivot back)

11. (6 pts) List a common sort that has the same (exact) run-time on any array of a given size, regardless of the initial ordering of the elements.

selection sort

merge sort

_____ Name a second one: _____

12. (5 pts) What is the big-theta runtime (in terms of m , n , and b) of radix-sorting a set of n integers with values in the range $[0, m)$ when using b bins?

$\Theta(n \log_b m)$ The base of the log depends on the number of bins you are working with, since $\text{ceil}(\log_b m)$ will give the number of digits in the numbers.

13. (6 pts) Given an extended binary tree (EBT) of height h , **including** any links to external nodes in the calculation of the height,

...what is the *maximum* number of external nodes it may have? **2^h**

...what is the *minimum* number of external nodes it may have? **$h+1$**

14. (8 pts) Describe (in words and/or diagrams) the **set of all** trees whose preorder and postorder traversals visit the nodes in opposite order.

The set of trees such that each node has exactly one child (except for the single leaf, which has no children).

Why? Pre: Node, L, R. Post: L, R, Node, post-backwards = Node, R, L. To let the left and right swap, no node can have 2 children.

15. (4 pts) If a height-balanced tree has height h , give a recurrence for the *minimum* number of nodes it can have. Then solve that recurrence to find a closed-form solution. (Note: we did this on the day 12 quiz even before we understood the term *recurrence*.)

Recurrence: **$S_h = S_{h-1} + S_{h-2} + 1$, where $S_{-1} = 0$ and $S_0 = 1$.**

Solution:

H	S
-1	0
0	1
1	2
2	4
3	7
4	12
N	$\text{Fib}(H+3)-1 = (\text{phi1}^{H+3} - \text{phi2}^{H+3})/\text{sqrt}(5) - 1$, where $\text{phi1} = (1+\text{sqrt}(5))/2$ and $\text{phi2} = (1-\text{sqrt}(5))/2$

I did this via “finding the pattern” since the Master method clearly doesn’t apply.